



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A06_CPE Caso práctico extendido – Prof. Emilio
Ramírez**

Autor: Segura Abad, Mario

Fecha: 11 de Noviembre, 2025

ÍNDICE

INTRODUCCIÓN 3

ENUNCIADO..... 4

GESTOR DE GASTOS: FILTRADO Y CÁLCULO DE AGRUPACIONES DE GASTOS 4

DESARROLLO DE LA PRÁCTICA..... 5

DESCRIPCIÓN..... 5

CONTENIDO DEL ARCHIVO..... 5

CASO PRÁCTICO EXTENDIDO – FILTRADO Y AGRUPACIÓN DE GASTOS..... 5

CLAVES DE RESOLUCIÓN Y PRUEBAS 6

- SE HAN DEFINIDO LAS FUNCIONES VACÍAS INICIALMENTE PARA EVITAR ERRORES DE SINTAXIS. 6
- SE HAN EJECUTADO LOS TESTS CON NPM RUN TEST3 PARA VALIDAR CADA FUNCIONALIDAD. 6
- SE HA UTILIZADO FILTER Y REDUCE PARA IMPLEMENTAR LÓGICA FUNCIONAL Y EVITAR BUCLES TRADICIONALES. 6
- SE HAN INCLUIDO PANTALLAZOS CON LOS RESULTADOS POR CONSOLA COMO PRUEBA DE AUTORÍA. 6

CÓMO VISUALIZARLO 7

REQUISITOS: 7

PASOS: 7

OBJETIVOS DE LA PRÁCTICA..... 7

- IMPLEMENTAR FUNCIONES DE FILTRADO Y AGRUPACIÓN DE GASTOS. 7
- APLICAR PROGRAMACIÓN FUNCIONAL CON FILTER Y REDUCE. 7
- MEJORAR LA ESTRUCTURA DEL OBJETO GASTO CON MÉTODOS PERSONALIZADOS. 7
- DOCUMENTAR CORRECTAMENTE, **COMO SIEMPRE**, CADA EJERCICIO CON NOMBRE, FECHA Y CAPTURAS DE PANTALLA. 7
- PRESENTAR EL PROYECTO EN FORMATO PDF JUNTO CON LOS ARCHIVOS FUENTE Y PANTALLAZOS..... 7

CONCLUSIÓN 8

INTRODUCCIÓN

En el ejercicio “Gestor de gastos: filtrado y cálculo de agrupaciones de gastos” deberemos descargar los archivos del proyecto para poder completar el caso práctico extendido y ejemplificar los contenidos del apartado 6 de la teoría de JavaScript.

ENUNCIADO

Gestor de gastos: filtrado y cálculo de agrupaciones de gastos

Caso práctico que ejemplifica los contenidos.

Descargad los archivos de proyecto para completar el caso práctico extendido.

Ejemplo de implementación:

```
JS gestionPresupuesto.js ×
DWE_C_U02_A04_z_CPE_E > js > JS gestionPresupuesto.js > CrearGasto
1  // Autor: Mario Segura Abad
2  // Fecha: 05/11/2025
3
4
5  // Variables globales
6  let presupuesto = 0;
7  let gastos = [];
8  let idGasto = 0;
9
10 // Funciones para Presupuesto
11 function actualizarPresupuesto(valor) {
12   if(typeof valor === "number" && valor >= 0) {
13     presupuesto = valor;
14     return presupuesto;
15   }
16   console.error("Valor inválido.");
17   return -1;
18 }
19
20 function mostrarPresupuesto() {
21   return `Tu presupuesto actual es de ${presupuesto} €`;
22 }
23
24 // Función constructora de gasto
25 function CrearGasto(descripcion, valor = 0, fechaStr, ...etiquetas) {
26   let fecha = Date.parse(fechaStr);
27   fecha = isNaN(fecha) ? Date.now() : fecha;
28
29   const gasto = {
30     descripcion,
31     valor: typeof valor === "number" && valor >= 0 ? valor : 0,
32     fecha,
33     etiquetas: [],
34
35     actualizarDescripcion(nuevaDescripcion) {
36       this.descripcion = nuevaDescripcion;
37     },
38   };
39 }
```

DESARROLLO DE LA PRÁCTICA

Práctica DWEK – Unidad 6

Descripción

Este caso práctico amplía las funcionalidades del gestor de presupuesto personal desarrollado en ejercicios anteriores. El objetivo principal ha sido implementar las máximas herramientas posibles de filtrado y agrupación de gastos para facilitar búsquedas específicas y análisis por periodos temporales. Se ha trabajado sobre el archivo `gestionPresupuesto.js` y se han ejecutado los tests con `npm run test3` para verificar el correcto funcionamiento del código.

Contenido del archivo

Caso práctico extendido – Filtrado y agrupación de gastos

1. Filtrado de gastos

Se ha creado la función `filtrarGastos`, que permite obtener un subconjunto de gastos según criterios como:

- Rango de fechas (`fechaDesde`, `fechaHasta`)
- Rango de valores (`valorMinimo`, `valorMaximo`)
- Texto en la descripción (`descripcionContiene`)
- Etiquetas asociadas (`etiquetasTiene`)

La función utiliza programación funcional (`filter`) y realiza comparaciones sin distinguir mayúsculas/minúsculas.

2. Agrupación de gastos

Se ha creado la función `agruparGastos`, que permite agrupar gastos por:

- Día (`aaaa-mm-dd`)
- Mes (`aaaa-mm`)
- Año (`aaaa`)

La agrupación se realiza mediante `reduce`, sumando los valores de los gastos que coinciden con el mismo periodo. Se pueden aplicar filtros por etiquetas y fechas para delimitar el conjunto de datos a agrupar.

3. Método obtenerPeriodoAgrupacion

Se ha añadido al objeto `Gasto` el método `obtenerPeriodoAgrupacion`, que devuelve el periodo correspondiente según la fecha del gasto y el tipo de agrupación solicitado.

Ejemplo de ejecución:

```

1) Filtra los gastos (variable global 'gastos') de acuerdo con los parámetros pasados a la función

Función agruparGastos
  ✓ Realiza una agrupación de gastos por períodos temporales

2 passing (153ms)
1 failing

1) Función filtrarGastos
  Filtra los gastos (variable global 'gastos') de acuerdo con los parámetros pasados a la función:

  Las opciones 'descripcionContiene', 'valorMinimo' o 'valorMaximo' no funcionan correctamente.
  + expected - actual

  -0
  +1

    at Context.<anonymous> (file:///Users/mariossegura/Desktop/Actividades%20y%20exa%CC%81menes/Todas%20actividades%202%C2%BA%20CFS_DAW/Desarrol
lo%20WEB%20en%20Entorno%20Cliente%20(Emilio)/TEMA%202%20-%20JavaScript/EJERCICIOS-TAREAS/U02_A06_CPE%20Caso%20pra%CC%81ctico%20extendido/DWEC_U02_
A04_z_CPE_E/test/03_testIntroJSIII.js:55:16)
    at process.processImmediate (node:internal/timers:485:21)

mariossegura@MacBook-Air-de-Mario DWEC_U02_A04_z_CPE_E % clear
mariossegura@MacBook-Air-de-Mario DWEC_U02_A04_z_CPE_E % npm run test3

> practica_dwec_gestor_presupuesto@1.0.0 test3
> mocha test/03_testIntroJSIII.js

Función CrearGasto y funcionamiento de objeto gasto
  ✓ Método 'obtenerPeriodoAgrupacion' del objeto gasto

Función filtrarGastos
  1) Filtra los gastos (variable global 'gastos') de acuerdo con los parámetros pasados a la función

Función agruparGastos
  ✓ Realiza una agrupación de gastos por períodos temporales

2 passing (63ms)
1 failing

1) Función filtrarGastos
  Filtra los gastos (variable global 'gastos') de acuerdo con los parámetros pasados a la función:

  Las opciones 'descripcionContiene', 'valorMinimo' o 'valorMaximo' no funcionan correctamente.
  + expected - actual

  -0
  +1

    at Context.<anonymous> (file:///Users/mariossegura/Desktop/Actividades%20y%20exa%CC%81menes/Todas%20actividades%202%C2%BA%20CFS_DAW/Desarrol
lo%20WEB%20en%20Entorno%20Cliente%20(Emilio)/TEMA%202%20-%20JavaScript/EJERCICIOS-TAREAS/U02_A06_CPE%20Caso%20pra%CC%81ctico%20extendido/DWEC_U02_
A04_z_CPE_E/test/03_testIntroJSIII.js:55:16)
    at process.processImmediate (node:internal/timers:485:21)

mariossegura@MacBook-Air-de-Mario DWEC_U02_A04_z_CPE_E %

```

Claves de resolución y pruebas

- Se han definido las funciones vacías inicialmente para evitar errores de sintaxis.
- Se han ejecutado los tests con `npm run test3` para validar cada funcionalidad.
- Se ha utilizado `filter` y `reduce` para implementar lógica funcional y evitar bucles tradicionales.
- Se han incluido pantallazos con los resultados por consola como prueba de autoría.

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Node.js: para ejecutar los tests con `npm run test3`.
- Navegador web (opcional): para visualizar los resultados si se integra con una interfaz HTML, que no es mi caso.

Pasos:

1. Guardar el archivo `gestionPresupuesto.js` en la raíz del proyecto, **muy importante este paso**.
2. Abrir la terminal en esa carpeta.
3. Ejecutar los tests con: `npm run test3`.
4. Verificar que se superen absolutamente todos los tests y que las funciones devuelven los resultados esperados.

Objetivos de la práctica

- Implementar funciones de filtrado y agrupación de gastos.
- Aplicar programación funcional con `filter` y `reduce`.
- Mejorar la estructura del objeto Gasto con métodos personalizados.
- Documentar correctamente, **como siempre**, cada ejercicio con nombre, fecha y capturas de pantalla.
- Presentar el proyecto en formato PDF junto con los archivos fuente y pantallazos.

CONCLUSIÓN

Esta práctica nos ha permitido consolidar el uso de programación funcional en JavaScript para gestionar y analizar datos.

Se ha trabajado con filtros condicionales, aplicando la lógica declarativa y evitando bucles tradicionales.

Además, se ha reforzado el uso de pruebas automáticas (`npm run test3`) como herramienta de verificación, y se ha aplicado documentación técnica para una entrega más clara y entendible.

La práctica ha sido clave para entender cómo estructurar una librería funcional y reutilizable.